Rules Decide, Language Models Narrate: A Fail-Closed Multi-Agent Architecture for Flood Response with Numeric Grounding

Nandhakumar T
Department of Information Technology
Sona College of Technology
Salem, Tamil Nadu, India
nandhakumartm93@gmail.com

Balakarthikeyan B S
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
balasan2626@gmail.com

Gowthaman J
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
gowthamanjanadhran@gmail.com

Abhishek S
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
abisheksengodan18@gmail.com

***Abstract*—Large language models (LLMs) are being proposed as the voice of emergency-management software, but they can state a flood probability, a river level or a head-count that is not in the data. We describe a multi-agent flood-response architecture in which this risk is contained by design: eleven role-specific agents share one state, every action is chosen by a deterministic policy, and the LLM is only allowed to narrate. Three mechanisms keep the system fail-closed. A gated policy downgrades automated response to human review when data, model or database health checks fail. A dual-model gate holds dispatch when a Random Forest and an XGBoost classifier disagree. A numeric grounding guard rejects any generated message that contains a number not traceable to the current state, and falls back to templates. We evaluate all three offline on the deployed code. Exhaustive testing of the policy over 16,032 input combinations found 0 violations of five safety properties. An audit of the dispatch path then showed that dispatch read the probability rather than the policy action, which broke the guarantee in 2,807 of 32,032 end-to-end cases; after a one-line fix, the same exhaustive check found 0. Across 604 monsoon rainfall settings the dual-model gate held dispatch in 69.7% of cases, including 73.7% of cases in which both models were above the automation threshold. Driven through the real generation code with 1,200 injected wrong numbers, the guard rejected 96.0% of them with no false rejections but missed every wrong value written in words. A slot-typed prototype raised detection to 98.8% and caught 81.5% of number words.**

***Index Terms*—multi-agent systems, agentic AI, large language models, hallucination, human-in-the-loop, fail-safe design, flood response**

I. Introduction

During a flood, an emergency operations centre runs on short spoken messages: the river is at this level, the forecast says this, send a team there. Software that supports the centre is therefore increasingly asked to *speak*, and LLMs make this easy. They are also known to produce confident statements that are not supported by their input [1]. In most applications a wrong number in generated text is an inconvenience. On an operations net it can trigger or cancel an evacuation.

Agentic systems built on LLMs go further and let the model plan and call tools [2]–[4]. For a safety-critical setting we take the opposite position. The decisions in our system are made by small deterministic agents whose behaviour can be tested exhaustively, and the LLM is restricted to turning their state into readable radio traffic. Two problems remain: the narrator can misreport the state, and an automated step can fail towards action instead of towards a human. Our position follows the classical view that the level of automation should be chosen per function and per risk [5], and that safety is a property of the control structure rather than of any single component [6].

This paper describes and evaluates that architecture as it is implemented in an open flood decision-support prototype for the Delhi Yamuna corridor. Its contributions are:

1) A multi-agent architecture in which agents share a single state and event log, actuation rights belong only to deterministic agents, and generative output is advisory.
2) Three fail-closed mechanisms (a health-gated decision policy, a dual-model disagreement gate and a numeric grounding guard for LLM messages), each defined formally.
3) An offline evaluation of all three on the deployed code: exhaustive property checking of the policy, a sensitivity analysis of the disagreement gate, and an injection study of the grounding guard that runs through the real prompt-building and parsing path.
4) A slot-typed guard that binds each number to the quantity it describes, evaluated against the deployed guard on the same injected errors.
5) Findings that matter for practice: the dispatch path bypassed the verified policy until we fixed it, the dis-

agreement gate is too strict for the two deployed models, and untyped number matching misses values that collide with other facts or are written in words.

II. Related Work

Agents and agentic LLMs. Software agents that perceive, decide and act autonomously have a long history [7], and disaster response has served as a multi-agent benchmark since RoboCup Rescue [8]. Recent work lets LLMs interleave reasoning with actions [2], learn when to call tools [3], cooperate in multi-agent conversations [4] and simulate believable human behaviour [9]. Surveys describe these systems as a memory, a planner and an action interface around a language model [10]. Our agents keep the first and last elements but take planning away from the model.

Hallucination and its detection. Ji *et al.* survey the ways generated text departs from its source [1]. Retrieval augmentation grounds generation in documents [11], and post-hoc checkers score factual precision by decomposing text into atomic claims [12] or by sampling the model several times [13]. These methods target open-domain facts and are probabilistic. Our setting is narrower: the facts are a handful of numbers in a known state, so a deterministic check is possible and cheap.

Safe automation. Parasuraman *et al.* separate information acquisition, analysis, decision and action, and argue for lower automation where errors are costly [5]. Run-time shields block unsafe actions of learned controllers [14], and concrete AI-safety problems include unsafe exploration and distributional shift [15]. Ensembles are a standard way to estimate when a model is uncertain [16], [17]. We use disagreement between two classifiers as a hold signal rather than averaging them.

III. Architecture

Fig. 1 shows the system. A FastAPI backend runs one pipeline per tick, in live mode (Open-Meteo weather and river data) or in simulation mode (a digital twin of the corridor). Eleven agents (Weather, Dam, Flood Risk, Traffic, Shelter, Evacuation, Rescue, Ambulance, Administrator, Disaster Team and a Card Monitor that watches for sudden jumps in displayed values) share one state object. Each agent keeps a small memory (status, last event, current action) and writes every event to a MongoDB log and a WebSocket stream, so the interface shows exactly what each agent did and why. Table I lists the agents, the events they emit and whether they hold any authority over actions. Only the Flood Risk agent's policy can start a response, and the dispatch step that calls the Rescue and Ambulance agents executes only what the policy allowed. Every other agent informs, ranks or narrates.

Algorithm 1 gives one pipeline tick as it is now implemented. The policy and the disagreement gate run before any planning agent, and narration runs last, after the decision has been logged, so a failure in narration can delay a radio message but cannot change what was decided. The dispatch line was changed after the audit in Section V-D; it previously read only the probability band.

TABLE I
Agents, their events and their authority

Agent	Emits	Authority
Weather	rainfall update	inform
Dam	river and barrage stage update	inform
Flood Risk	policy action π and reason	**decide** (gated)
Traffic	road risk and blockages	inform
Shelter	ranked shelters with capacity	rank
Evacuation	candidate routes, evacuation plan	propose
Rescue	team dispatch	act after π
Ambulance, Disaster Team	radio turns	narrate
Administrator	hold notices, radio turns	hold / narrate
Card Monitor	sudden-change alerts	inform

Algorithm 1 One pipeline tick (after the fix)

read observations; Weather and Dam agents emit updates
$p_{\mathrm{RF}}, p_{\mathrm{XGB}}, p \leftarrow$ classifiers; $H \leftarrow$ (2)
$(h_d, h_m, h_s) \leftarrow$ health checks; $a \leftarrow \pi(p, \mathbf{h}, o)$ by (1)
if H **and not** o **then** $a \leftarrow$ REVIEW and log the hold reason
if $a \neq$ MONITOR **then**
    Traffic, Shelter, Evacuation agents rank roads, shelters, routes
end if
if $a \in \{\text{AUTO}, \text{OVERRIDE}\}$ **and** band(p) = auto **then** dispatch teams
$T \leftarrow$ LLM turns for band(p); **if not** $\Gamma(T)$ **then** $T \leftarrow$ template
broadcast state, decision and T

A. Decision Policy

The Flood Risk agent receives a probability p of flooding from the policy model and three health flags: h_d (weather and river data available), h_m (model loaded and returned a value) and h_s (database connected). An operator can set an override o, and a configuration flag A enables automation. With a monitor threshold $\tau_m = 0.50$ and an automation threshold $\tau_a = 0.60$, both served from a single configuration module, the policy is

$$\pi(p, \mathbf{h}, o) = \begin{cases} \text{UNAVAILABLE} & p \text{ undefined} \\ \text{OVERRIDE} & o \\ \text{MONITOR} & p < \tau_m \\ \text{AUTO} & p \geq \tau_a \wedge h_d \wedge h_m \wedge h_s \wedge A \\ \text{REVIEW} & \text{otherwise.} \end{cases} \tag{1}$$

The last case covers both the grey zone $\tau_m \leq p < \tau_a$ and any high-risk state in which a health check failed. The policy is a pure function, so its safety properties can be checked over its whole input space (Section V-A).

B. Dual-Model Disagreement Gate

The same features are scored by a Random Forest [18] and an XGBoost classifier [19], giving p_{RF} and p_{XGB}. Automatic dispatch is held when

$$H = \big[\, |p_{\mathrm{RF}} - p_{\mathrm{XGB}}| \geq \delta \,\big] \\ \vee \ \big[\min(p_{\mathrm{RF}}, p_{\mathrm{XGB}}) < \tau_a \leq \max(p_{\mathrm{RF}}, p_{\mathrm{XGB}}) \big], \tag{2}$$

with $\delta = 0.15$. The first term flags a large gap; the second flags the case where the two models fall on opposite sides of the automation threshold even with a small gap. When H

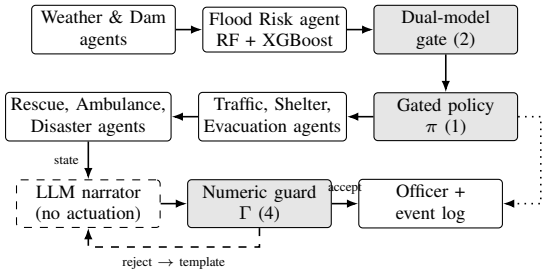


Fig. 1. Agent architecture. Shaded blocks can only hold, downgrade or reject; none of them can trigger an action. The LLM (dashed) receives the shared state and has no actuation rights. The dotted link carries every policy decision and its reason to the officer.

holds, the Administrator agent announces the hold and the Rescue agent stays staged until the models agree or an operator overrides.

C. LLM Narration and the Numeric Grounding Guard

Agent conversations are generated by an LLM served through an external API, with several open-weight models tried in order. The conversation band follows the probability: calm below 0.40, watch below 0.50, administrator review below 0.60 and automated response above, and a disagreement hold forces the administrator band. The prompt lists the current facts (probability, rainfall, river level, barrage stage, model scores, time and place), states the band's rule, and instructs the model to use only those numbers. The reply must be JSON with speaker, addressee and text. A parser canonicalises agent names and drops malformed turns.

Every reply then passes a guard. Let $\mathcal{N}(s)$ be the decimal numbers in a string s, and let the fact sheet F be the prompt text up to the recent-history block. The admissible set is

$$\tilde{F} = \{\, f,\ 100f,\ f/100\ :\ f \in \mathcal{N}(F)\,\}, \tag{3}$$

so that 0.734 and 73.4% are both accepted. A set of turns T is accepted when

$$\Gamma(T) = \bigwedge_{t \in T}\ \bigwedge_{n \in \mathcal{N}(t)}\ \exists f \in \tilde{F} :\ |n - f| \le \tfrac{1}{2}10^{-d(n)}, \tag{4}$$

where $d(n)$ is the number of decimals written in n. The tolerance accepts honest rounding (73 for 73.4) and nothing more. A rejected reply is retried with the next model and, if all fail, replaced by a template filled from the same state. The prompt shows recent radio lines for continuity but marks them as possibly stale, and they are excluded from F so that an old value cannot be re-validated.

The guard is untyped: it checks that a number exists among the facts, not that it is used for the right quantity. If a hallucinated value n is drawn uniformly from an integer range $[a, b]$, its chance of passing is approximately

$$P_{\text{miss}} \approx \frac{|\{\, \lfloor f \rceil : f \in \tilde{F}\,\} \cap [a, b]|}{b - a + 1}, \tag{5}$$

which grows with the size of the fact sheet. Numbers written as words are invisible to $\mathcal{N}$ and always pass.

TABLE II
NARRATION BANDS, SPEAKERS AND PROMPT RULES

Band (p)	Speakers	Rule given to the LLM
calm (< 0.40)	Weather, Flood Risk	reassure; call if it rises
watch (0.40–0.50)	Flood Risk, Administrator	watch only; no rescue
admin (0.50–0.60)	Administrator, Flood Risk	no automation; hold rescue
auto (≥ 0.60)	Rescue, Ambulance, Disaster	send teams to named places
spike	Card Monitor, Flood Risk	report the metric that jumped

A disagreement hold forces the admin band whatever the value of p.

Table II lists the bands, the agents that speak in each and the rule the prompt gives them. The bands only choose who speaks and what tone the narration takes; the action itself is still set by π.

D. A Slot-Typed Guard (Prototype)

To test whether these weaknesses can be removed cheaply, we built a typed variant Γ_{T} that is evaluated offline and is not yet deployed. It reads facts from the structured state instead of the prompt text and binds each number in a message to a quantity before checking it. Number words ("sixty") are first rewritten as digits. Clock times must equal the current time. A number followed by "%" or "percent", or preceded by "probability", must match a model probability or a threshold of the current band; one followed by "mm" or preceded by "rain" must match the rainfall; and one followed by "m" must match the river level or barrage stage. A number that cannot be bound to any of these quantities is rejected, because the state contains no other numeric facts:

$$\Gamma_{\mathrm{T}}(T) = \bigwedge_{t \in T}\ \bigwedge_{(n,k) \in \mathcal{B}(t)}\ \exists f \in F_k :\ |n - f| \le \tfrac{1}{2}10^{-d(n)}, \tag{6}$$

where $\mathcal{B}(t)$ pairs each number with its slot k and F_k holds the facts of that slot. Any number for which $\mathcal{B}$ finds no slot fails the check.

IV. Evaluation Setup

All experiments run offline against the deployed backend modules, with fixed seeds, on a consumer laptop.

Policy. We evaluated (1) on $p \in \{0, 0.001, \ldots, 1\}$ plus an undefined value, all eight health combinations and both override states: 16,032 cases. We checked five properties: (P1) AUTO only if every gate passes and $p \geq \tau_a$; (P2) undefined p gives UNAVAILABLE; (P3) $p < \tau_m$ without override gives MONITOR; (P4) $p \geq \tau_a$ with any failed check gives REVIEW; (P5) with all checks passing, the action never decreases as p rises.

Dual-model gate. We scored both deployed classifiers for 24-h rainfall from 0 to 300 mm in 2-mm steps, for each monsoon month from June to September (604 settings), and applied (2).

Grounding guard. We drew 200 states with seeded rainfall between 5 and 250 mm and a monsoon month, took both model probabilities from the deployed classifiers, and sampled river level (200.0–208.5 m) and barrage stage (199.0–206.0 m) from the ranges used by the twin. The banding put 21, 8, 10

TABLE III
ACTIONS OF THE POLICY OVER THE 16,032 CHECKED CASES

Action	Cases
MONITOR	4,000
REVIEW	3,607
AUTO	401
OVERRIDE	8,008
UNAVAILABLE	16

Half of all cases carry an operator override by design of the grid.

and 161 states in the calm, watch, administrator and automated bands. For each state we called the production function that builds the prompt, queries the API, parses the reply and applies Γ, with the HTTP client replaced by a local stub that returns a prepared reply. No request left the machine. Each state was tested with one grounded reply (four turns quoting the probability, a rounded probability, rainfall, river level, barrage stage and time) and with seven corrupted copies, each containing one wrong value of a given type (Table V). The fact sheets contained 8.1 distinct numbers on average. We also re-ran the matcher on the captured fact sheets with two changes: without the $\times 100/\div 100$ expansion, and with only the facts line (excluding the numbers in the band rules). The typed prototype (6) was applied to the same replies with the same states.

Dispatch path. Finally, we audited the pipeline code that connects the policy to dispatch. We then extracted the dispatch decision into a single pure function and checked the composed path, policy, disagreement hold and dispatch, over the same grid of probabilities, health flags, holds and overrides (32,032 cases), for the property that teams are dispatched only when every health check passes and the models agree, or when an operator overrides. The check was run with the original dispatch rule and with the corrected one.

V. RESULTS

A. Policy Properties

None of the 16,032 cases violated P1–P5, and the check took 42 ms. Table III shows how the cases were distributed over the five actions.

The policy returned AUTO in 401 cases, all with every check passing, and REVIEW in 3,607 cases. No high-risk case with a failed check reached AUTO. This result is expected for a five-line function. We keep the decision logic small for this reason: the property then holds for every input, including inputs that no test scenario covers.

B. Dual-Model Gate

Fig. 3 plots the two classifiers for July and shades the rainfall settings in which the gate held dispatch. Over all four months, the Random Forest exceeded τ_a in 79.5% of settings and XGBoost in 80.8%. The gate held dispatch in 69.7% of settings, and in 76.9% of those where at least one model exceeded τ_a. The gap term fired in 64.1% and the straddle term in 10.3%, but the straddle term fired without a large gap in only 5.6%. The mean gap between the models was 0.167, which is larger than the tolerance $\delta = 0.15$ itself, and the largest gap was 0.347.

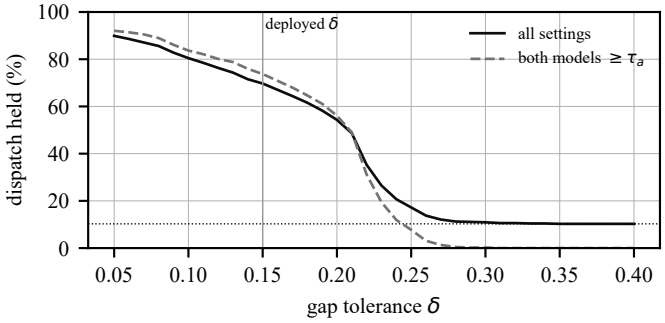


Fig. 2. Share of rainfall settings in which the gate (2) holds dispatch, as a function of the gap tolerance δ. The dotted line is the straddle-only rule.

TABLE IV
GATE SENSITIVITY TO THE GAP TOLERANCE δ (SAME 604 SETTINGS)

δ	Held (%)	Held when both $\geq \tau_a$ (%)
0.10	80.5	83.7
0.15	69.7	73.7
0.20	54.3	56.1
0.25	17.2	7.7
0.30	10.9	0.2
0.35	10.3	0.0
straddle only	10.3	0.0

The consequence is that the gate rarely separates genuine disagreement from routine disagreement. Of the 453 settings in which *both* models exceeded τ_a, 73.7% were still held because their probabilities differed by more than 0.15. In those settings both models recommend action, so holding is overly conservative. It is safe in the narrow sense, because a hold falls back to human review, but an officer who sees holds most of the time will learn to override them.

Table IV recomputes the gate on the same sweep for other tolerances. Raising δ to 0.30 cut the hold rate to 10.9% and the share of held both-above-threshold cases to 0.2%. Keeping only the straddle term would hold 10.3% of settings and never hold a case in which both models recommend action. Fig. 2 shows the same quantities on a finer grid. Both curves fall steeply between $\delta = 0.20$ and 0.25, which shows that most gaps between the two models lie below 0.25, and the share of held both-above-threshold cases first reaches zero at $\delta = 0.31$. Below that value, the tolerance mostly measures how differently the two models are calibrated, not whether they disagree about the decision.

The straddle-only rule matches the purpose of the gate more closely: it holds exactly when the choice between review and automation depends on which model is trusted. A tolerance, if kept, should be set from the gap distribution on validation data rather than chosen by hand.

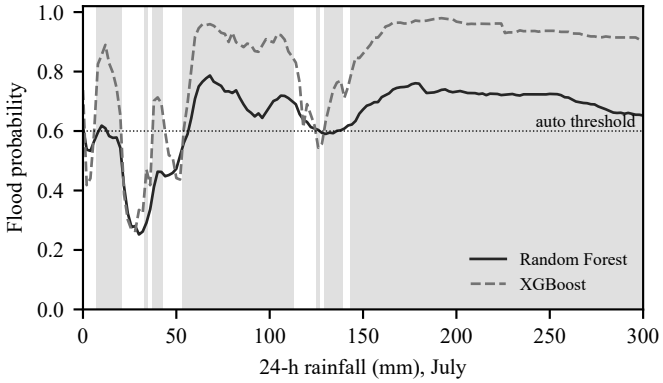


Fig. 3. Random Forest and XGBoost flood probability against 24-h rainfall for July. Shaded rainfall values are those for which the gate (2) held automatic dispatch.

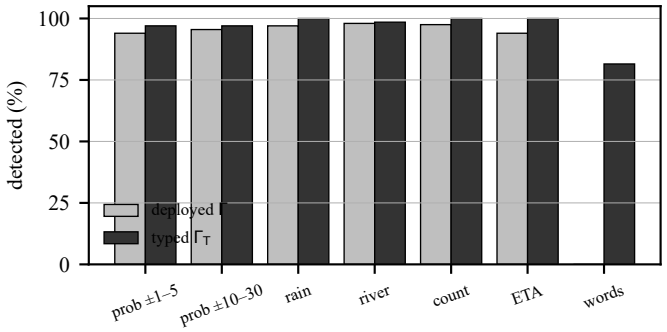


Fig. 4. Share of injected wrong values detected, by error type, for the deployed guard (4) and the typed prototype (6).

C. Numeric Grounding Guard

Table V gives detection rates. The guard accepted all 200 grounded replies, including those with honestly rounded values, so there were 0 false rejections. Over the 1,200 replies with a wrong digit string, it rejected 96.0%. Each stubbed call through the production path took about 2.1 ms, so the guard adds nothing noticeable to a radio update.

The 4.0% of misses were not random. We traced each of the 48 missed values to the admissible number it matched: 39 coincided with a genuine fact used for a different quantity (for example, 10 invented ETAs equalled the rounded flood probability), 8 equalled a threshold quoted in the band rule, and 1 matched only a $\times 100$ or $\div 100$ copy. This is the behaviour predicted by (5). A typical case: with a flood probability of 87.3%, a reply that invented an "ETA 87 minutes" passed, because 87 is within the rounding tolerance of the probability, which is a legitimate fact. Fig. 4 compares the deployed and typed guards for every error type.

Removing the scale expansion or the rule numbers raised detection only to 96.8%, because most collisions are with real facts rather than with the expansions. The remaining weakness is structural: the guard verifies that a number exists, not that it is attached to the right quantity. Wrong values written in words ("about sixty percent") passed in every case.

TABLE V
GUARD DETECTION RATE (%) BY INJECTED ERROR TYPE, 200 STATES EACH

Injected error	Depl.	No scale	Facts only	Both	Typed
Probability ±1–5 pts	94.0	94.0	95.0	95.0	97.0
Probability ±10–30 pts	95.5	95.5	96.5	96.5	97.0
Rainfall 0.5–1.5×	97.0	97.0	97.5	97.5	100.0
River level ±0.5–2 m	98.0	98.5	98.0	98.5	98.5
Invented head-count	97.5	97.5	98.0	98.0	100.0
Invented ETA	94.0	94.0	95.0	95.0	100.0
All numeric errors	96.0	96.1	96.7	96.8	98.8
Wrong value in words	0.0	0.0	0.0	0.0	81.5

Grounded replies accepted: 200/200 in every column. Typed = prototype (6).

The typed prototype (6) addressed both problems. It accepted 200 of 200 grounded replies (0 false rejections) and rejected 98.8% of the numeric errors, with complete detection of invented head-counts and ETAs, because those numbers have no slot in the state. It also caught 81.5% of the wrong values written in words. Its 15 remaining numeric misses were values that coincided with another fact of the same slot, such as the other model's score, a threshold of the current band or the other water level, all of which are legitimate values for that slot. The number words it missed matched a band threshold or a model score in the same way. Typed binding does not make the check complete, since a message could still swap two quantities of the same slot, but it removes the cross-quantity collisions that caused most misses.

D. End-to-End Audit of the Dispatch Path

The property check covers the policy function, so we also traced how its output is used. Two gaps appeared in the original pipeline. First, the dispatch step tested the probability band, $p \geq \tau_a$, and the disagreement hold, but it did not read the action a returned by π. If a health check failed while both models agreed on a high probability, π correctly returned REVIEW, yet teams were still dispatched. Second, the disagreement hold was applied only in simulation mode, so in live mode disagreement was reported but did not stop dispatch.

We fixed both. Dispatch now requires the policy action to be AUTO or OVERRIDE, and the hold applies in both modes. With the original rule, the exhaustive end-to-end check found 2,807 violating cases out of 32,032, all of them high-probability states with a failed health check, in which teams would have been sent without review. With the corrected rule it found 0. Teams were dispatched in 9,624 cases before the fix and 6,817 after it, so the fix removed exactly the unsafe dispatches. Three regression tests now guard the rule in the backend test suite. The episode shows that fail-closed properties have to be checked on the whole path that triggers action, including the code after the function that names the action.

E. Deployment Note: A Silent Failure of the LLM Path

While preparing this evaluation we found that the LLM path of the deployed prototype had not produced a single accepted reply. Two independent faults were responsible. The

configured chat models had been retired by the API provider, so every request to them returned HTTP 404. The fallback models were reasoning models whose hidden reasoning counts against the output budget; with the 700-token limit of the original code, they ran out of tokens before writing the required JSON, and the provider returned HTTP 400 with a validation error. In both cases the narrator fell back to its templates, so the radio kept working and the operators saw sensible messages. The failure was therefore invisible from the interface.

We fixed the request builder: it now asks reasoning models for brief reasoning, raises their output budget, and lists only current models. A single reply generated after the fix, for a state with probability 73.4%, 120.5 mm of rain, a river level of 204.2 m and a barrage stage of 203.1 m at 14:05, quoted all five values exactly and was accepted by both guards. One reply is an illustration, not a measurement, and a systematic study of live replies remains future work. The episode is still instructive: a fail-closed fallback hides the failure it absorbs, so the fallback rate itself has to be monitored and reported.

VI. Discussion and Limitations

The results support a division of labour. Where the decision logic is small and deterministic, it can be verified completely, and LLM failures cannot reach the decision. Where the LLM does contribute, its contribution can be checked cheaply because the facts it may use are known. The weak points we found are in the glue between components: a dispatch step that bypassed the verified policy until we fixed it, a disagreement tolerance that was never calibrated against the models it compares, and a guard that treats every number as interchangeable. Each of them sits between components rather than inside one, so testing one component at a time does not reveal it. For agentic systems in safety-critical settings, this argues for checking properties on the full path from observation to action, and for keeping the number of components with authority to act as small as possible.

There are also threats to the validity of the measurements themselves. The injected errors follow seven types we chose, so the detection rates describe these types and not the error distribution of a real model. Each state was tested with one grounded reply written to resemble agent traffic; replies with more numbers would give the untyped guard more chances to collide. The dual-model sweep varies rainfall and month only and uses the default forecast inputs of the deployed function, so it shows the calibration gap between the two models but not how often operators would meet it in a real monsoon.

Several limits apply. We did not measure how often real LLM replies fail the guard, because live calls were outside the scope of a reproducible offline study, and the injected errors are our own construction rather than a sample of model mistakes. River and barrage values in the guard study were sampled from operating ranges, not taken from recorded events. The guard checks numbers only, so a wrong place name, a wrong agent or a wrong instruction would pass. The policy check proves properties of (1), not of the health checks that feed it, and the end-to-end check covers the dispatch rule on a grid of inputs rather than a formal proof of the asynchronous pipeline. The typed guard is a prototype whose slot rules were written for this fact sheet, so new quantities would need new rules. Finally, no officer has used the system in an exercise, so we cannot yet say whether the narration helps or distracts.

Future work will log and alarm on the template-fallback rate, deploy the typed guard, replace the gap tolerance with the straddle rule or a calibrated δ, record live LLM replies to estimate the real rejection rate, and run a table-top exercise with emergency officers.

VII. Conclusion

We presented a flood-response architecture in which deterministic agents hold every decision and an LLM only narrates. Exhaustive testing found no violation of five safety properties of the decision policy, but the dispatch path did not consult it. After a one-line fix, an end-to-end check found no violation in 32,032 cases, which shows that such properties must be checked on the full path to action. The dual-model gate held dispatch in 69.7% of monsoon settings and in 73.7% of settings where both models already recommended action, and a straddle-only rule would remove those unnecessary holds. The numeric guard caught 96.0% of injected wrong numbers with no false rejections. A slot-typed version raised this to 98.8% and also caught most numbers written in words. Narration from an LLM can be useful on an operations net, provided it stays separate from the decision path and is checked against the state it describes.

Acknowledgment

The authors thank the INDOFLOODS dataset authors and Open-Meteo for openly available data. In line with the IEEE policy on AI-generated content, the authors disclose that Claude (Anthropic) was used to help edit and restructure the text of all sections and to draft the experiment and plotting scripts. All experiments use the authors' own software, run offline and can be reproduced with fixed seeds. The authors checked every number, claim and reference and take full responsibility for the content.

References

[1] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023.

[2] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[3] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Adv. Neural Inf. Process. Syst. 36*, 2023.

[4] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[5] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 30, no. 3, pp. 286–297, 2000.

[6] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. Cambridge, MA, USA: MIT Press, 2011.

[7] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *Knowl. Eng. Rev.*, vol. 10, no. 2, pp. 115–152, 1995.

[8] H. Kitano *et al.*, "RoboCup Rescue: Search and rescue in large-scale disasters as a domain for autonomous agents research," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Tokyo, Japan, 1999.

[9] J. S. Park *et al.*, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST)*, 2023, Art. no. 2.
[10] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, Art. no. 186345, 2024.
[11] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst. 33*, 2020, pp. 9459–9474.
[12] S. Min *et al.*, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 12076–12100.
[13] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 9004–9017.
[14] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 2669–2678.
[15] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," arXiv:1606.06565, 2016.
[16] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems* (LNCS, vol. 1857). Berlin, Germany: Springer, 2000, pp. 1–15.
[17] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Adv. Neural Inf. Process. Syst. 30*, 2017, pp. 6402–6413.
[18] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
[19] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.